

1) `[10 + num for num in range(1,6)]`

<code>num =</code>	<code>num =</code>	<code>num =</code>	<code>num =</code>	<code>num =</code>
<code>[1,</code>	<code>2,</code>	<code>3,</code>	<code>4,</code>	<code>5]</code>
<code>[10+num</code>	<code>10+num</code>	<code>10+num</code>	<code>10+num</code>	<code>10+num]</code>
<code>[11,</code>	<code>12,</code>	<code>13,</code>	<code>14,</code>	<code>15]</code>

2) `[n <= 4 for n in [4, 9, 2, 7, -1]]`

<code>n=</code>	<code>n=</code>	<code>n=</code>	<code>n=</code>	<code>n=</code>
<code>[4,</code>	<code>9,</code>	<code>2,</code>	<code>7,</code>	<code>-1]</code>
<code>[n <= 4</code>	<code>n <= 4</code>	<code>n <= 4</code>	<code>n <= 4</code>	<code>n <= 4]</code>
<code>[4<=4</code>	<code>9<=4</code>	<code>2<=4</code>	<code>7<=4</code>	<code>-1<=4</code>
<code>[True,</code>	<code>False,</code>	<code>True,</code>	<code>False,</code>	<code>True]</code>

3) `[idx for idx in range(len(L)) if L[idx] > 5]` `L = [5, 3, 8, 1, 6]`

<code>idx=</code>	<code>idx=</code>	<code>idx=</code>	<code>idx=</code>	<code>idx=</code>
<code>[0,</code>	<code>1,</code>	<code>2,</code>	<code>3,</code>	<code>4]</code>
<code>L[idx]>5</code>	<code>L[idx]>5</code>	<code>L[idx]>5</code>	<code>L[idx]>5</code>	<code>L[idx]>5</code>
<code>5>5</code>	<code>3>5</code>	<code>8>5</code>	<code>1>5</code>	<code>6>5</code>
<code>False</code>	<code>False</code>	<code>True</code>	<code>False</code>	<code>True</code>
<code>[</code>		<code>2</code>	<code>,</code>	<code>4]</code>
<code>[2,4]</code>				

4) `[n for n in [4, 9, 2, 7, 21] if n % 3 == 0]`

n=	n=	n=	n=	n=
[4,	9,	2,	7,	21]
n%3==0	n%3==0	n%3==0	n%3==0	n%3==0
4%3==0	9%3==0	2%3==0	7%3==0	21%3==0
False	True	False	False	True
[	9,			21]
[9,21]				

5)

```

1 def inc(n:int) -> int:
2     return n + 1
3
4 [inc(x) for x in range(4) if x%2 != 0]
```

	Call <code>inc(1)</code> -> 2
	Known Bindings
Line 1	<code>n=1</code>
Line 2	<code>n=1</code>

	Call <code>inc(3)</code> -> 4
	Known Bindings
Line 1	<code>n=2</code>
Line 2	<code>n=2</code>

6)

```

1 def check(value:int) -> bool:
2     return value %3 != 0
3
4 [number for number in [-1, 2, -3, 4, -5, 9] if check(number)]

```

	Call <code>check(-1)</code> -> True
	Known Bindings
Line 1	<code>value = -1</code>
Line 2	<code>value = -1</code>

	Call <code>check(2)</code> -> True
	Known Bindings
Line 1	<code>value = 2</code>
Line 2	<code>value = 2</code>

7)

```
1 def validate(value:int) -> bool:
2     return value > 0
3
4 def square(x:int) -> int:
5     return x * x
6
7 [square(number) for number in [-1, 2, -3, 4, -5, 9] if
validate(number)]
```

	Call <code>square(2) -> 4</code>
	Known Bindings
Line 1	<code>value = 2</code>
Line 2	<code>value = 2</code>

	Call <code>square(4) -> 16</code>
	Known Bindings
Line 1	<code>value = 4</code>
Line 2	<code>value = 4</code>

8)

```
1 def tally(values:list[int]) -> int:
2     accumulator = 0
3     for idx in range(0, len(values), 2):
4         accumulator = accumulator + values[idx]
5     return accumulator
6
7 result = tally([1, 4, 2, 9, 3])
```

Lists
<pre>newList = [1,4,2,9,3]</pre>

	Known Bindings
Line 7	<pre>result = 6</pre>

	Call <code>tally(newList)</code> ->6
	Known Bindings
Line 1	<pre>values = newList</pre>
Line 2	<pre>values = newList, accumulator = 0</pre>

Iteration	1	2	3
Line 3	<pre>idx = 0, accumulator = 0</pre>	<pre>idx = 2, accumulator = 1</pre>	<pre>idx = 4, accumulator = 3</pre>
Line 4	<pre>idx = 0, accumulator = 1</pre>	<pre>idx = 2, accumulator = 3</pre>	<pre>idx = 4, accumulator = 6</pre>